

January 2026 CSE 406: Cyber Security Sessional

Online 2: Ethical Hacking

Section: C2

Time: 40 minutes

Ethical-use warning

This is an intentionally vulnerable, isolated teaching environment. Use the techniques in this assignment only against the supplied local containers. Never test an application, account, or network without explicit authorization. The same actions against a real service may be unethical and illegal.

Objective

You have access to two campus applications: the **Food Review App** and **Student Chat**. Acting only as the designated attacker account, fully compromise the pair as follows:

Discover the structure of the Food Review database and extract every student's existing food rating. Then persist a browser payload through the same vulnerable input, in your own name, so that logging in normally automatically publishes a convincing food-review link in Student Chat. Finally, demonstrate that when a victim views the chat and clicks that link while logged into the Food Review App, a CSRF sets their rating to exactly 5/5.

Environment Setup

The Moodle archive contains `C2-images.tar`, `compose.yaml`, and this specification. Internet access is not required after download. From the directory containing those files, run:

```
seed@ubuntu:~$  
  
docker load --input C2-images.tar  
docker compose up -d  
docker compose ps
```

Wait until all three services are running. Open:

- Food Review App: `http://localhost:9000`
- Student Chat: `http://localhost:9001`

To reset every database change, session, and chat message and return to the original seeded state:

```
seed@ubuntu:~$  
  
docker compose down  
docker compose up -d
```

The 40-minute limit includes image loading and environment startup. Raise your hand immediately if Docker reports an infrastructure error before either page opens.

Credentials

Role	Student ID	Food Review login	Student Chat login
Attacker	9900001	rafid.bari / bari-99	rafid.bari / chat-Rafid-99
Victim	9900002	mahin.kader / kader-99	mahin.kader / chat-Mahin-99

Tasks and Marks

The core assignment is worth 10 marks. One bonus mark is available.

1. **SQL injection confirmation (1 mark):** Demonstrate that the Food Review App login can be made to authenticate without knowing a correct password.
2. **Schema discovery (2 marks):** Enumerate the relevant application tables and columns and identify how a student's rating and review are stored.
3. **Complete rating extraction (2 marks):** Extract every student's ID, name, rating, and review text through the vulnerable login form.
4. **Persistent browser payload (1 mark):** Use a stacked statement through the login input to persist a stored-XSS payload in the attacker's own Food Review record.
5. **Food Review App to Student Chat compromise (2 marks):** With the attacker logged in to Student Chat, a normal Food Review App login must automatically create a Student Chat post containing a convincing food review link, with no separate manual post.
6. **Final CSRF (2 marks):** With the victim logged in to the Food Review App, clicking the food review link posted in Student Chat must change the victim's rating to exactly 5/5, driven only by the victim's own authenticated session.
7. **Bonus - cover your tracks (+1):** Discover the forensic trail left by Food Review App login submissions and, through the same injection channel, delete only the trail attributable to the attacker account. Evidence belonging to every other student must remain intact.

Restrictions

- All injection, extraction, persistent modification, and cleanup used for grading must be issued through the Food Review App login form while acting as the attacker account, **9900001**. Direct database edits do not earn marks.
- The victim credentials are supplied only so you can confirm the final CSRF from the victim's point of view. Logging in as the victim to read their rating is not extraction, and submitting the victim's student ID directly to any endpoint does not earn CSRF credit.
- Do not attack anything outside the supplied local environment.
- Do not disrupt another student's workstation or containers.

Submission

Create a plain-text file named with your seven-digit student ID, for example 2105XXX.txt. For every graded stage, include:

1. the exact value or payload submitted;
2. the application, field, and account where it was submitted;
3. the observed result and all extracted ratings/reviews; and
4. the ordered steps needed to reproduce the final rating change.

Submit only this text file through Moodle. Payloads without their input location or without enough context to reproduce the attack may not receive credit.